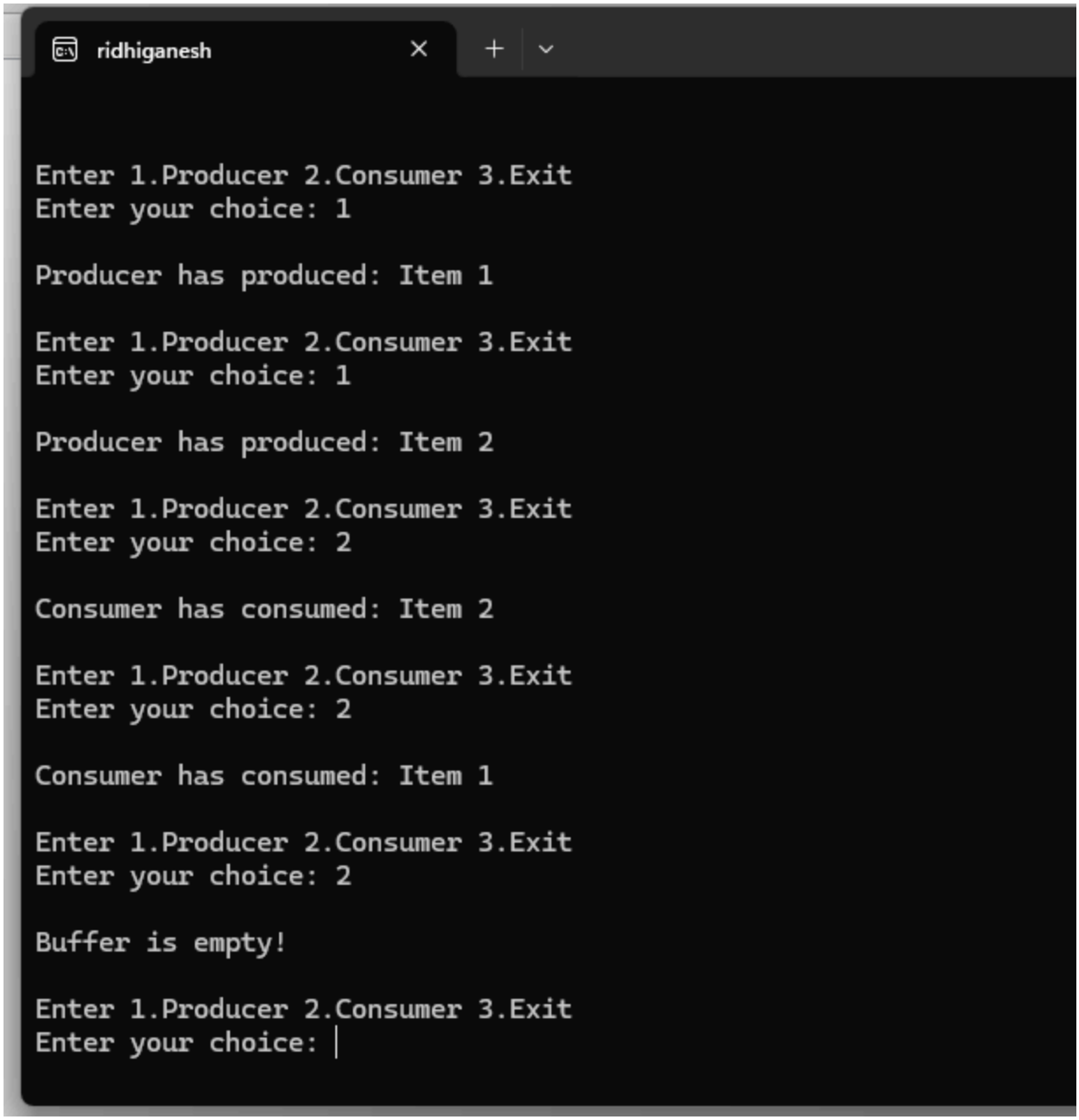
Week 3

```
#include <stdio.h>
#include <stdlib.h>

// Global variables
int mutex = 1;
int full = 0;
int empty = 5, x = 0; // Buffer size is 5

// Function to signal/release a semaphore
int signal(int s) {
return (++s);
}

// Function to wait/decrement a semaphore
int wait(int s) {
return (--s);
}

void producer() {
// Decrease empty slots, increase mutex to lock
empty = wait(empty);
mutex = wait(mutex);

x++;
printf("\nProducer has produced: Item %d", x);

// Release mutex and increase full slots
mutex = signal(mutex);
full = signal(full);
}

void consumer() {
// Decrease full slots, decrease mutex to lock
full = wait(full);
mutex = wait(mutex);

printf("\nConsumer has consumed: Item %d", x);
x--;

// Release mutex and increase empty slots
mutex = signal(mutex);
empty = signal(empty);
```

```
}

int main() {
int n;

while (1) {
printf("\n\nEnter 1.Producer 2.Consumer 3.Exit");
printf("\nEnter your choice: ");
scanf("%d", &n);

switch (n) {
case 1:
// Check if buffer is full (empty slots == 0)
if ((mutex == 1) && (empty != 0)) {
producer();
} else {
printf("\nBuffer is full!");
}
break;

case 2:
// Check if buffer is empty (full slots == 0)
if ((mutex == 1) && (full != 0)) {
consumer();
} else {
printf("\nBuffer is empty!");
}
break;

case 3:
exit(0);
break;

default:
printf("\nInvalid Choice!");
}
}
return 0;
}
```



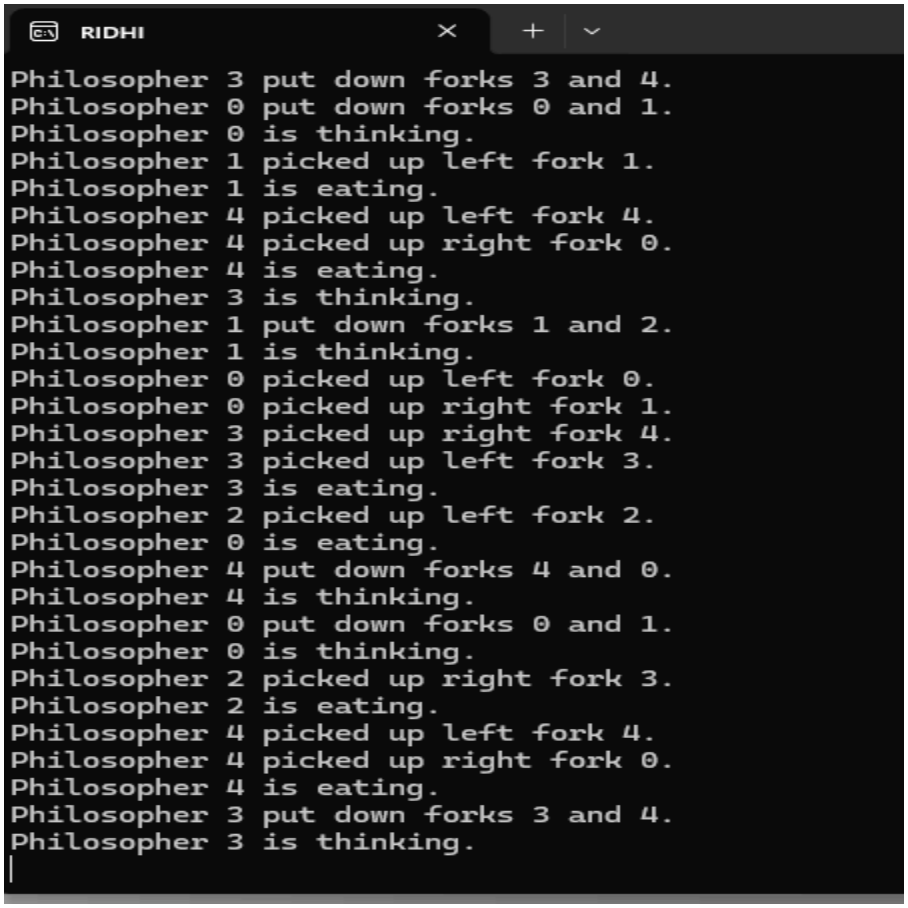
1WA24CS229

2. SENOPHORE

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5 // Number of philosophers
```

```
pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];

void* philosopher(void* num)
{
    int id = *(int*)num;
    int left = id;              // left fork index
    int right = (id + 1) % N;   // right fork index

    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);

        if (id % 2 == 0)
        {
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);

            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
        }
        else
        {
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);

            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }

        printf("Philosopher %d is eating.\n", id);
        sleep(2);

        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);

        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
    }

    return NULL;
}

int main()
```

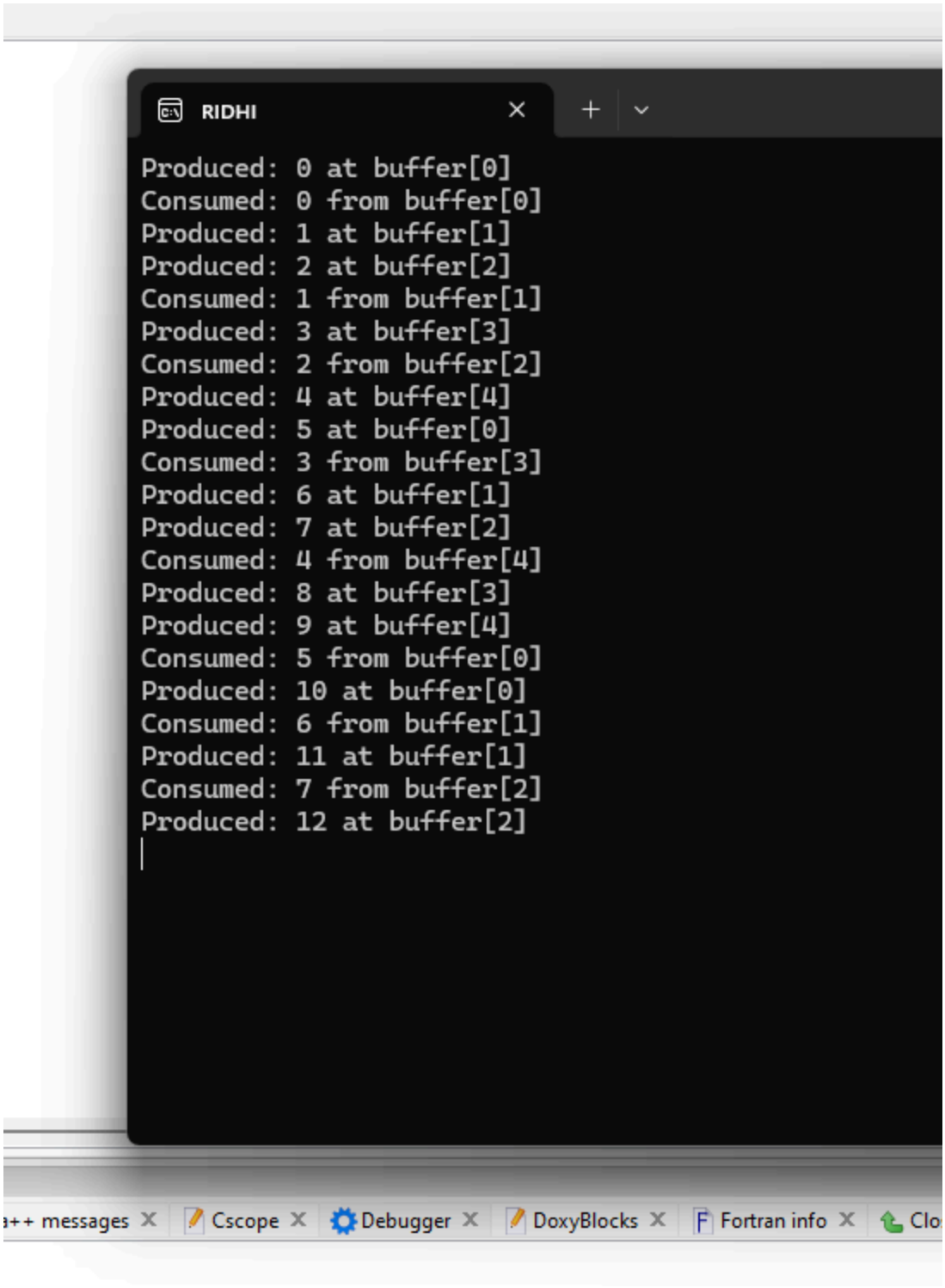
```
{
    int i;
    int ids[N];

    for (i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    for (i = 0; i < N; i++)
    {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    for (i = 0; i < N; i++)
    {
        pthread_join(philosophers[i], NULL);
    }

    for (i = 0; i < N; i++)
    {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}
```



PRODUCER CONSUMER WITHOUT QUEUES

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#include<semaphore.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty; // counts empty slots
sem_t full; // counts filled slots
pthread_mutex_t mutex; // for mutual exclusion
void* producer(void* arg) {
int item, i = 0;
while (1) {
item = i++; // produce an item (here just incrementing number)
```

```
sem_wait(&empty); // wait for empty slot
pthread_mutex_lock(&mutex); // enter critical section
buffer[in] = item;
printf("Produced: %d at buffer[%d]\n", item, in);
in = (in + 1) % BUFFER_SIZE;
pthread_mutex_unlock(&mutex); // leave critical section
sem_post(&full); // signal that new item is available
sleep(1); // simulate time to produce
}
}
void* consumer(void* arg)

{
int item;
while (1)
{
sem_wait(&full); // wait for filled slot
pthread_mutex_lock(&mutex); // enter critical section
item = buffer[out];
printf("Consumed: %d from buffer[%d]\n", item, out);
out = (out + 1) % BUFFER_SIZE;
pthread_mutex_unlock(&mutex); // leave critical section
sem_post(&empty); // signal that a slot is free
sleep(2); // simulate time to consume
}
}
int main()
{
pthread_t prod, cons;
sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&full, 0, 0); pthread_mutex_init(&mutex, NULL);
pthread_create(&prod, NULL, producer, NULL);
pthread_create(&cons, NULL, consumer, NULL);
pthread_join(prod, NULL);
pthread_join(cons, NULL);
sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);
return 0;
```


RIDHI
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Produced: 2 at buffer[2]
Consumed: 1 from buffer[1]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[1]
Consumed: 7 from buffer[2]
Produced: 12 at buffer[2]
++ messages
Cscope
Debugger
DoxyBlocks
Fortran info

1WA24CS229